Assignment 1: Do personas annotate like people, and does context change them?

🗓️ Deadline	👤 Work	🛠️ Tools
02/11/2026	individual	Python, Colab (T4 GPU), SynDisco

💡 The idea

Is this comment toxic? Is this review sarcastic? Careful readers can disagree, and none of them is simply wrong. For these **subjective** tasks there is no ground truth, so we ask people to **annotate**: to read items and label them. Their labels are not the truth, they are a **reference point**, and their disagreement is information, not only noise.

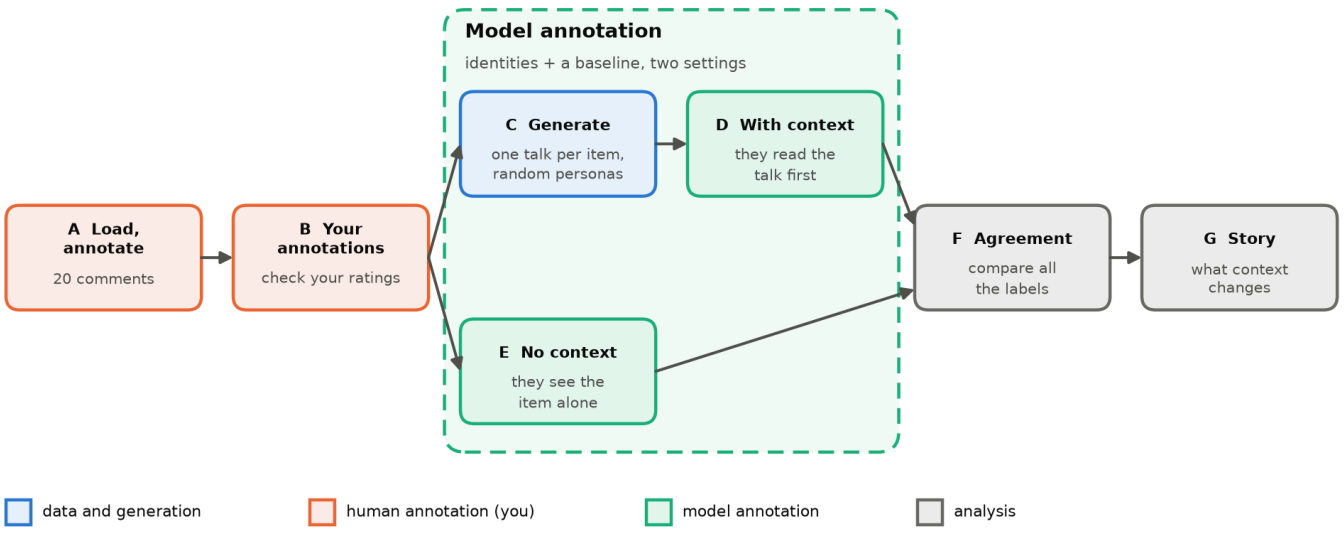
But annotation is slow and costly: you have to find and train people, and wait for their labels, and reading toxic content is unpleasant. So more and more teams use a language model as the annotator, often with a **persona** ("you are a 45-year-old conservative woman...") so that it labels like a given group. A model also never reads a text in a vacuum: what it has just read, such as a conversation about the item, can change its answer.

To run this study you will use **SynDisco**, a Python library for synthetic discussions. It lets language models, each with a persona, hold a conversation about a text, and it lets other models, also with personas, annotate it.

A cheap label is useful only if it looks like what a real person would say. So you annotate 20 items yourself, as the reference, then use SynDisco to get persona annotators that label the same items alone and after reading a conversation, and compare yourself with them.

> 🎯 **When a model plays a person, do its labels look like those of real people? And what changes when it first reads a conversation about the text?**

This is the pipeline you will build, in seven parts. You first annotate your own items (A, B). Then a model generates a conversation about each item (C), and persona annotators label the items with that conversation as context (D) and without it (E). Finally you measure how close they are to you (F) and tell the story (G).



Each part has a task and something you hand in:

Part	You do	You hand in
A	Draw **your own** 20 comments with the script, then annotate them yourself in the small web page we give you	your seed, your 20 items and ratings (with evidence for the toxic ones), shown in the notebook
B	Open your items and your annotations in your notebook, and show how you rated	a histogram and a boxplot of your ratings
C	Generate one small conversation about each item	the 20 conversations, shown in the notebook
D, E	Persona annotators rate the items like you did, with the conversation as context (D) and alone (E)	the tables of annotations of both conditions, shown in the notebook
F	Measure simple agreement across everything	the agreement tables and figures, in the notebook
G	Compare the two settings and tell the story	written explanations in markdown cells of the notebook

> 📦 **You upload only two files: your notebook (with all your explanations written in markdown cells inside it) and a markdown file with your interactions with LLMs.** See *Hand in and grading* at the end.

> 💻 **Part A runs on your own computer** (Python 3 and a browser). **From part B on, use Google Colab with a T4 GPU**: it is free, and the model runs much faster than on a CPU (*Runtime → Change runtime type → T4 GPU*).

📥 A. Load and annotate

Two steps, in this order: **load** your own sample of comments with a script, then **annotate** it in a small web page. Do both **before you look at any model's labels**, and do not change your ratings afterwards.

Step 1. Load: draw your own sample

> ⚠️ **Content note:** the comments may contain harsh language, insults, slurs and violent wording. That is what you are asked to rate.

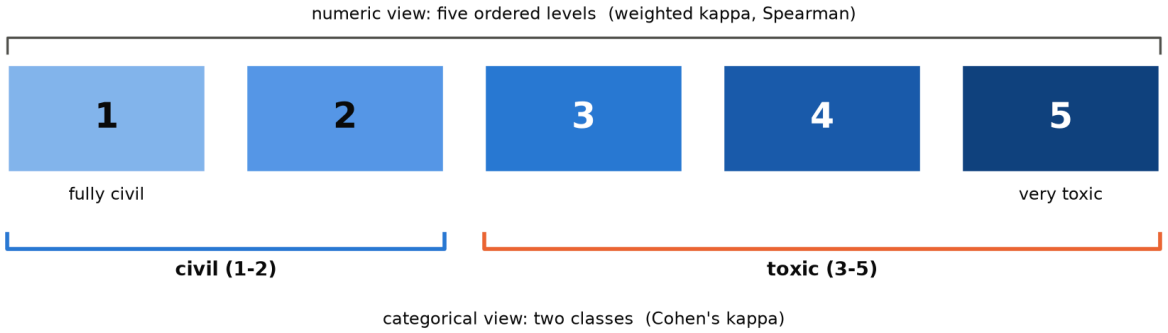
You get a folder with the script `prepare_items.py` and `data/pool.csv`: 250 short comments from news-site discussions, from fully civil to very toxic. Each student rates **their own sample of 20** of them, which depends on the **seed** you choose. From that folder, run the script once, with your own seed (for example your student id):

```
pip install pandas
python prepare_items.py --seed 123456
```

The first line installs **pandas**, which the script needs (only the first time). The second line runs the script, which writes `items/items.csv` (your 20 items) and `items/annotate.html` (the page of step 2).

- **Seed:** the same seed gives the same 20 items, another seed gives another sample. Write it down in the first cell of your notebook, and **keep the same seed**: if you change it after you have started rating, you get other items and your ratings no longer match.
- **Range:** your sample mixes civil and toxic texts, so rate each one on its own.
- You do not need to open `data/pool.csv`: the script reads it for you.

Step 2. Annotate: rate the 20 items in the page



Open `items/annotate.html` in your browser (double-click it; it works offline), read the guidelines page, and click *Start annotating*. For each of the 20 items:

1. click a **rating** from **1 (fully civil) to 5 (very toxic)**;
2. if you rated **3, 4 or 5**, **select with the mouse the words that make it toxic**: they are saved as your **evidence**. For a 1 or a 2 the evidence is **not needed**: you only give the rating, and the page writes `not needed` in the evidence column.

Next unlocks when the item has a rating (and, for 3-5, at least one selection). The page also adds a `label` by itself: 1-2 are `civil`, 3-5 are `toxic`. That rule is ours, not your choice, and we use it later for the civil/toxic comparison.

Your answers are saved in the browser, so you can close the page and come back (same browser; do not clear the site data). After the 20th item, **Download annotated CSV** appears: it gives you `items_rated.csv` with the columns `id | text | rating | label | evidence`.

✍️ B. Your annotations in the notebook

Upload `items.csv` and `items_rated.csv` to Colab and open both in your notebook. `items.csv` holds the 20 texts you will give to the models (parts C, D and E), and `items_rated.csv` holds your annotations. Then show **how you rated** with a few figures:

- a **histogram** of your ratings (how many 1s, 2s ... 5s);
- a **boxplot** of your ratings;
- optionally, anything else that shows how you used the scale (for example how many items are civil and how many toxic).

Say in a sentence or two what they show: did you use the whole scale, and do you lean towards civil or toxic?

`items_rated.csv` is your reference point for part F: keep it as it is. Since you upload only the notebook, **print both tables in full in the notebook** (the 20 items and your ratings, labels and evidence), so that they can be read and checked without your files.

💬 C. Generate one conversation per item

For each of your 20 items (a comment), random participants with different identities have a short conversation about it, with the item as their starting context. The how is explained in the SynDisco guide Creating discussions.

What you must have: 20 conversations that make sense, one per item. A conversation makes sense when the participants react to the item, in their own voice, without cutting off in the middle of a sentence, refusing, or repeating each other. **Show them in your notebook** (all 20, readable).

You choose the settings (how many participants, how many comments, how long each comment may be). Keep the size moderate: too short gives the annotators little to read, too long is slow and may drift away from the item.

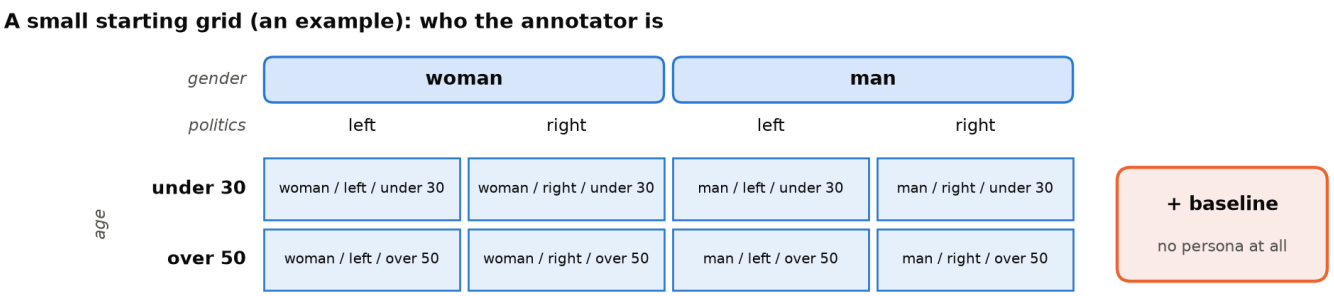
> 💡 **Hints**
>
> - One discussion per item. All participants must know the item: the guide shows how to give them a shared context.
> - Here you **may** guide the participants (their stance, how heated they are...) to get a varied, useful conversation: it is only context. This is the opposite of D and E, where a persona says only who it is.
> - The model gives the same output for the same input: the variety comes from your random participants (fixed seed).

🧩 Two uses of identities: the grid

Identities appear **twice** in this assignment, for **two different** jobs:

	C: conversations	**D and E: annotation**
Which identities	a pool of random identities, only to make the conversations (here you **may** guide how they talk)	the identities of the **grid**, chosen by you
Who acts	a few of the pool talk about each item, not all every time	**every** persona of the grid annotates **every** item, with context (D) or without (E)

A **persona** describes **who the annotator is** (gender, politics, age...). In SynDisco it is a dictionary of attributes that goes into the annotator's prompt. You do not tell it how to rate: the instructions about the label are the same for every annotator, and the model decides how a person like that would rate. That is exactly what we test. A **grid** lists a few values for each dimension and combines them all. Each cell is one persona, and each persona is one annotator.



Each cell is one persona, and each persona becomes one annotator. This grid is only a starting point: you will adapt it.

The grid above is only an example to start from. This is the persona the model would receive for two of its cells:

Cell	Persona (attribute: value)
woman / left / under 30	`{"gender": "woman", "politics": "left", "age": "under 30"}`
man / right / over 50	`{"gender": "man", "politics": "right", "age": "over 50"}`
baseline	`{}`: no persona (what the model does by default)

> 🧩 **You must change or extend this grid.** It is only a starting point, and using it unchanged is not accepted. Add a dimension (education, religion, origin...), add a value ("non-binary", "center"), or change the attributes and values of the personas, and say why. Keep it small, since every cell annotates all 20 items in two conditions. Write down the exact attributes and values each persona gets.
>
> ⚠️ **Describe who the annotator is, never how it should rate.** No "strict", "lenient" or "rate harshly": that would be you deciding the labels, not the model. The instructions are the same for everyone (parts D and E); the identity and the context are the only things that change.

🧠 Which model?

A good small starting point is **Qwen2.5-3B-Instruct** (4-bit): fast, but weak, it may refuse, ramble or ignore the format. **You choose the model**: this one, one from the list below, or any other that your computer or Colab can handle. The list is only a suggestion. Run your notebook with it. It is good practice to **try a few models** on a few items before you choose, to get the best result: a small model often gives poor conversations, and a larger one may follow the instructions better. You may also run a second model on the whole task to see whether your results hold. Say in the notebook which model you used and why. The ones below are pre-shrunk to 4 bits, need no account or token, and fit the 16 GB of a Colab T4:

Model (`model_path`)	Size	Download	Licence	Why try it
`unsloth/Qwen2.5-3B-Instruct-bnb-4bit` (**starting point**)	3B	2.1 GB	Apache 2.0	The default: small and fast, but the weakest
`unsloth/Qwen2.5-7B-Instruct-bnb-4bit`	7B	5.5 GB	Apache 2.0	Same family, larger: what does size change?
`unsloth/Llama-3.1-8B-Instruct-bnb-4bit`	8B	5.7 GB	Llama 3.1 community	A strong general model from another family

Model (`model_path`)	Size	Download	Licence	Why try it
`unsloth/Phi-4-mini-instruct-bnb-4bit`	4B	2.9 GB	MIT	A different kind of small model, good with instructions

To use another one, change only `model_path` where you create the model:

```
model = TransformersModel(
    model_path="unsloth/Llama-3.1-8B-Instruct-bnb-4bit",  # the start was "unsloth/Qwen2.5-3B-Instruct-bnb-4bit"
    name="base_model",
    max_out_tokens=120,  # long enough for a comment; a rating needs far less
)
```

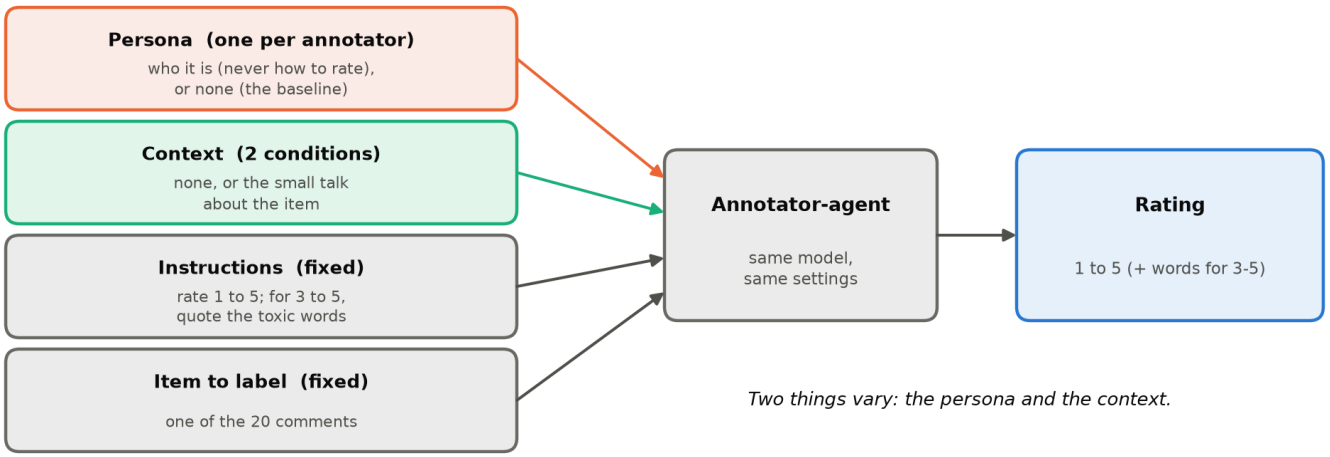
A larger model usually follows the instructions better but is slower (8B takes about twice as long per call as 3B, and you make hundreds of calls).

> 💡 **Hints**
>
> - **One model per run**, the same in D and E, for every persona and the baseline. To compare models, repeat the whole of D and E for each and say so in the notebook.
> - **Compatibility varies.** Models handle the system prompt and the output format differently. If a model fails every time (an error, or answers you can never parse), switch to another one. An occasional invalid answer is normal: count it, as in parts D and E.
> - **Prefer models that answer directly.** Reasoning models (for example the *Qwen3* family) write a long chain of thought first, which makes a short, parseable answer hard to get.
> - Load **one model at a time**: free the GPU (`del model`, restart the runtime if needed) before the next.

🤖 D and E. Annotate with and without context

There are two conditions. In **each** of them, every identity of your grid annotates every item, and so does the **baseline**, an annotator with no identity at all. It lets you tell whether a persona changes anything. So for each item and condition you get one label per persona, plus one from the baseline.

- **D, with context:** the annotator first reads the conversation generated for that item (part C), then labels **the item** (not the conversation).
- **E, no context:** the annotator sees only the item.



> 📌 **Requirement: D and E differ only in the context.** For the same annotator, the model, the settings, the items and the prompt are the same in D and E: same instructions, same persona, same wording. The only difference in the saved prompt is the `context` (the conversation in D, nothing in E). Your tables carry the prompt so that this can be checked.

The model annotators do the same task as you. For every item, each one gives a **rating from 1 (fully civil) to 5 (very toxic)**. If it rates **3, 4 or 5**, it also gives **the words of the item that make it toxic** (its evidence); for a 1 or a 2 the evidence is `not needed`. The `label` (1-2 `civil`, 3-5 `toxic`) comes from the rating, with the same rule as in the page.

Report the ratings of each condition exactly as people did it: one table for D and one for E, with the columns `id | text | rating | label | evidence` for every annotator, plus a column `annotator` to tell them apart and a column `prompt` with the prompt the annotator received (SynDisco saves it with every answer). Show both in your notebook, as one table with a column `context` or as two tables.

> 💡 **Hints**
>
> - Ask for the same thing you did: the rating and, for 3-5, the toxic words quoted from the item, in a fixed format you can parse (for example `4 | the words`). The answer is short, so keep the output limit small, unlike in part C. Expect the model not to always obey: parse the answers, **count and show the invalid ones** (no rating from 1 to 5, or words that are not in the item) per persona and condition, and do not fix them silently.
> - **How the context reaches the annotator.** In SynDisco an annotator is built from a `persona`, `instructions` and a `context`, and the library puts all three in its system prompt. It then goes through a log one entry at a time, and the annotator labels each entry. So think about which of the three should carry the conversation in D. And be careful with the log: whatever you put in it gets labelled, and you want the label of the **item**, not of the comments of the conversation. The annotation guide shows how an annotator is created.
> - Test on 2 annotators and 3 items first, and open one output file: the persona and the prompt are saved in it.

📊 F. Agreement

The story. Your ratings are the reference. Everything so far was meant to answer one question: *how close are the model annotators to you, and what changes that?* We answer it by comparing annotators on the same 20 items, in four steps, each with its own question:

	Compare	The question it answers
1	**you** against each persona and the baseline, **without context** (E)	Does a model that plays a person rate like a real person? Is any persona closer to you than the baseline is?
2	**you** against each persona and the baseline, **with context** (D)	Does reading the conversation bring the models closer to you, or further away?
3	each persona and the baseline, **without against with context** (E against D)	How much does the context move an annotator on its own? Is the baseline moved more or less than the personas?
4	**persona against persona**, within each condition (the baseline is one more annotator)	Does identity matter at all? Which dimension (gender, politics, age...) separates the personas the most? Do they agree more or less once they have read the conversation?

How to measure. Use only **simple agreement**: the share of the 20 items on which two annotators give the same answer (for example 14 of 20 = 70%). Compute it in **two ways**: **categorical** (same label, civil against toxic, 1-2 against 3-5) and **numerical** (same exact 1-5 rating). No chance-corrected measures (kappa, alpha) are needed. For persona against persona (comparison 4), compute the agreement of every pair and show it as a table or a heatmap, plus the average per condition. Interpret the values next to the results: a number alone is not a result.

> 🎯 **The key result: which identity annotates most like you?** Rank the personas by agreement with you, with and without context, and name the closest one **with all its attributes** (for example "closest to the *woman / left / over 50*"; if you added dimensions, name all of them). Is it closer to you than the baseline is? Does it change when the context is added?

> 💡 **Hints**
>
> - Simple agreement is one line of pandas or numpy: `(a == b).mean()` on two label columns sorted by id.
> - Match the items by id, and drop or count the invalid answers (say which you did). Check for an annotator that always gives the same label: its agreement then only reflects how many items have that label.
> - Simple agreement does not correct for chance: with two labels, 50% is what coin flips would give, and if most items are toxic a constant annotator already scores high. Keep that in mind when you interpret the values.
> - With 20 items the values are noisy (one item is 5 points): take that into account when you interpret them.

📝 G. The explanations, inside the notebook

There is no separate report. Write your explanations in **markdown cells of the notebook**, next to the code and results they belong to, and finish with a short closing section. Keep them concise: about **3-4 pages** of text in total. They have three parts.

> 🌐 **The explanations must be written in English.**

> 🔗 **The first cell of the notebook must contain your seed and the link to your GitHub repository.** Working with GitHub on this assignment is mandatory.

1. Justify your choices. At each step say what you did **and why**: your seed, how you rated (the guidelines you followed, an example of your evidence), how you made the conversations (the number of participants and of comments, the length limit, and why), the model you chose and why (and which others you tried, if any), your grid (what you changed or added compared with the starting grid, the dimensions and the exact attributes and values of each persona, and why), where you put the context, how you handled invalid answers.

2. Results and story. Report the results of the four comparisons of part F, in tables and figures. **Interpret** the agreement values: what they mean and what they tell you. This is the largest part. Does context move the labels, and in which direction? Which identity is closest to you, and is it the same with and without context? Does the context bring the models closer to you or further away? Do the personas agree more or less with each other once they have read the conversation? Which identities move the most? Do the personas differ from the baseline at all, and which dimension (gender, politics, age...) matters most?

> ⚖️ **State clearly that the closest identity does not mean you *are* that identity.** Think about the limitations: 20 items, a small model, one generated conversation per item (and that conversation is itself a bias), no sampling, personas that are only descriptions.

3. AI use and sources. State in a short section how you used AI (chatbots, code assistants...): for what, and what you checked yourself. The full record goes in the separate markdown file (see below). List your sources: the guides, papers and documentation you used, and anything else you relied on.

✅ Hand in and grading

You upload:

Exactly **two files**:

- ☐ **your notebook** (`.ipynb`), with the seed and the GitHub link in the first cell and **every explanation written in markdown cells inside it** (part G). It shows, in this order: your items and ratings (A, B), the 20 conversations (C), the grid with the exact attributes of each persona, the annotation tables with and without context for every persona plus the baseline on all 20 items, with the `annotator` and `prompt` columns (D, E), the agreement results (F), and the explanations (G).
- ☐ **a markdown file with your interactions with LLMs** (e.g. `llm_interactions.md`): the prompts you sent to chatbots or code assistants and their answers, or a faithful summary of each exchange if it is very long. Say at the top which tools you used. If you used no AI, write that in the file.

You do not upload `items.csv`, `items_rated.csv`, `generated.jsonl` or any other file: everything must be **visible in the notebook**. Choose the clearest form (printed output, tables, figures), with titles and labels. A result that is computed but not shown does not count: someone reading your notebook must see each result without running anything. The notebook should also run top to bottom.

Anything missing means it is not graded until complete.

Part	Weight
A	15%

Part	Weight
B	10%
C	20%
D, E	20%
F	20%
G	15%

> ❗ **Mandatory: every choice and every result in the notebook must be justified.**
>
> For each step, state **what** you did, **why** you did it, and **the result** you obtained. A choice without a reason, or a result without an explanation, will not be accepted. For example: "I measured agreement as the share of items with the same label because [reason]; the values obtained are [values]."

📖 What to read

Read these **before part C**: it is the tool you will use for the conversations and the annotators.

Read	Why
SynDisco overview	what the tool is and how its pieces fit together (personas, discussions, annotators)
Discussion guide	for part C: how random participants talk about an item, and how to give them a shared context
Annotation guide	for parts D and E: how an annotator is created, what it receives and what it writes
SynDisco paper	the research behind the tool: useful for the limits you discuss in your explanations

Then answer for yourself: what is a persona, and what does the model receive? What do all the participants of a discussion share? What does an annotation experiment take as input, and what does it write?